K: the programming language

Wojciech Fraczak
fraczak@gmail.com

October 2026

Abstract

K is a minimalist programming language inspired by the algebraic principles of Milner's CCS and Hoare's CSP. Computation is modeled as partial functions (called *relations*) over rooted, edge-labeled trees, built from three fundamental combinators: sequential composition, concurrent choice, and parallel composition. Together with elementary tree navigation and construction, these combinators replace the control-flow constructs, variable bindings, and built-in primitive types of traditional programming languages.

In K, both partial functions (relations) and their types (called *patterns*) are content-addressable. Because K is Turing-complete, semantic function equality is undecidable; however, patterns denote regular tree languages, which admit an effective canonical representation. Through parameterized *Named Patterns*, K factors schemas into canonical linear skeletons and parameter routing vectors, enabling exact structural type equivalence, global cryptographic deduplication, and static verification with zero runtime overhead.

Keywords: Relational programming, Process calculi, Content-addressable code, Regular tree languages, Type systems.

1 Introduction

K is a relational programming language inspired by the algebraic principles of Milner's Calculus of Communicating Systems (CCS) [1] and Hoare's Communicating Sequential Processes (CSP) [2]. Computation in K is modeled as first-order partial functions (called *relations*) over **rooted, edge-labeled trees**, structured around three fundamental algebraic composition modes:

1. **Transformation Chaining**: feeding the output tree of one relation as the input to the next (sequential composition).
2. **Concurrent Choice**: evaluating alternative relations and selecting the first one defined on the input (alternative composition).
3. **Parallel Composition**: evaluating multiple relations in parallel on the same input tree and assembling their results under distinct labeled branches of a new tree.

Together with elementary tree navigation and construction, these three combinators replace the disparate mechanisms of traditional programming languages—such as conditional branching, local variable bindings, argument passing, and built-in primitive types—within a single uniform algebra.

Within this framework, types (called *patterns*) are regular tree languages that can be embedded directly into relations as domain-restricting identity filters. The compiler verifies these structural contracts statically and uses them as inductive witnesses to ensure type derivation converges, leaving execution free of runtime type checks.

This work presents the K relational calculus (Section 2), its static pattern and type system (Section 3), **Named Patterns** and the Linear Canonical Theorem for global cryptographic content-addressing (Sections 4–6), and K's self-describing binary serialization format (Appendix A).

2 The Core Calculus: Schemas, Documents, and Relations

2.1 Documents and Schemas

K programs compute over **documents**, mathematically defined as finite, rooted, edge-labeled trees. Crucially, *a document is always interpreted in the context of a schema* (called a *pattern* in K)—a deterministic regular tree automaton [5] defining the language of admissible trees.

Schemas and documents are built from two dual structural constructors:

- **Products**: In the context of a product schema $\{S_1\ \text{label}_1, S_2\ \text{label}_2, \dots\}$, a product document consists of labeled outgoing edges pointing to sub-documents, each conforming to its respective sub-schema S_i. The empty product `{}` corresponds to the unique **unit document**.
- **Variants (Unions)**: In the context of a union schema $\langle S_1\ \text{tag}_1, S_2\ \text{tag}_2, \dots\rangle$, a variant document selects a tag tag_k and carries a payload document conforming to sub-schema S_k. A variant document only has meaning within a union schema specifying its admissible alternatives.

At system boundaries, every value is a self-describing pair of (schema, document).

2.2 Relations: Three Combinators

Computation in K is expressed as first-order partial relations that transform an incoming document into an outgoing document under their respective schemas. In addition to transformation chaining (functional composition), K adopts from process calculi (Milner's CCS and Hoare's CSP) two structural constructions: concurrent choice and parallel composition. In the following, $f_1, f_2, \dots, f_n$ denote K relations, $\text{label}_1, \text{label}_2, \dots$ denote outgoing edge labels, and $P_1, P_2, \dots$ denote the analogous process expressions in CCS and CSP:

1. **Transformation Chaining `(f1 f2 ...)`**: Chains relations such that the output document of f_1 is passed as input to f_2, and so on ($f_n \circ \cdots \circ f_1$, corresponding to *Sequential Composition* $P_1; P_2; \dots$ in CSP). An empty composition `()` denotes the identity relation.
2. **Concurrent Choice `< f1, f2, ... >`**: Evaluates alternative relations (corresponding to *Alternative Composition* $P_1 + P_2 + \dots$ in CCS, or $P_1 \square P_2 \square \dots$ in CSP): if f_1 is defined on the incoming document, its output is produced; otherwise, computation proceeds to f_2. An empty choice `<>` is undefined everywhere.
3. **Parallel Composition `{ f1 label1, f2 label2, ... }`**: Evaluates relations in parallel (corresponding to *Parallel Composition* $P_1 \mid P_2 \mid \dots$ in CCS, or $P_1 \parallel P_2 \parallel \dots$ in CSP) on the same incoming document x, constructing a product document whose labeled edges $\text{label}_1, \text{label}_2, \dots$ point to the respective results $f_1(x), f_2(x), \dots$. If any branch f_i is undefined, the entire product relation is undefined.

Elementary relations navigate and construct document trees:

- **Product projection `.label`**: traverses edge `label`, returning the sub-document.

- **Variant projection `/tag`**: traverses variant `tag`, returning its payload document if the tag matches.
- **Variant constructor `|tag`**: constructs a variant document by tagging the incoming document with `tag`.

Under these combinators:

- The empty product `{}` is a relation that transforms any input into the unit document.
- Constant relations combine `{}` with constructor relations: for example, `{} |true` produces the boolean true document, and `{} |zero` produces the Peano zero document.

3 Patterns: Compile-Time Prototypes and Inductive Witnesses

A **pattern** defines a regular tree language accepted by a deterministic tree automaton [3]. Syntactically, a pattern P is constructed from:

- **Product and union patterns**: $\{P_1\ l_1, \ldots, P_n\ l_n\}$ and $\langle P_1\ t_1, \ldots, P_n\ t_n \rangle$, which may optionally include a literal ellipsis `...` (e.g., `< P1 t1, ... >`) to denote an *open* product or union permitting additional unlisted fields or tags;
- **Wildcards and pattern variables**: bare identifiers `X`, which act as polymorphic pattern variables unifying all occurrences of `X`, and the wildcard `(...)`, which acts as an anonymous pattern variable;
- **Recursive bindings**: `P = X`, which binds the pattern node P to `X` so that occurrences of `X` inside P form cycles in the automaton.

For example, the pattern

```
< {} nil, { (...) car, List cdr } cons > = List
```

describes a linked list: a list is a variant that is either `nil` (carrying the empty product `{}`) or `cons` (carrying a product with a head element under label `car` and a recursive tail list `List` under label `cdr`). Because the cycle returns to the same `cons` node, all elements of the list share the same type.

3.1 Filters as Static Prototypes

In languages like ML, a first-order function f is typed via a separate signature $f : \text{In} \to \text{Out}$. In K, a pattern P is embedded directly into relation syntax as a filter `?P`—an identity relation whose domain and codomain are restricted to P—so a full signature is written simply as `(?In f ?Out)`. This uniform notation is particularly convenient for partial inline annotations: for example, in `(f1 f2 ?P f3)`, the filter `?P` witnesses only the output of `f2` and the input of `f3`. Because the compiler verifies all filter constraints statically, `?P` is a pure no-op at runtime, incurring zero execution overhead.

3.2 Inductive Witnesses in Contract Derivation

The K compiler derives a contract $P_{\text{in}} \to P_{\text{out}}$ for every relation via fixed-point iteration. Because elementary projections such as `/0` and `/+1` only infer *open* variant patterns (`< ... >`), and open nodes at different depths cannot be merged automatically by state minimization, an unannotated recursive definition such as

```
peel_bad = < /+1 peel_bad, /0 |0 >;
```

fails to converge, unrolling an infinite chain of nested open unions `< X1 0, < X1 0, < ... > +1, ... > +1, ... >`.

By inserting inline filter annotations sharing a pattern variable `X`:

```
peel = ?X < /+1 ?X peel, /0 |0 >;
```

the two occurrences of `?X` unify the relation's input domain with the payload of `/+1`. This explicitly witnesses the inductive cycle and allows type derivation to converge immediately to the polymorphic contract:

```
?< X1 0, X0 +1, ... > = X0  -->  ?< X1 0, ... >
```

4 Named Patterns and Parameterized Schemas

K provides named patterns as parameterized regular tree templates:

```
-- Arity-0 leaf patterns:
bit[]  = < {} 0, {} 1 >;
bits[] = < {} _, B 0, B 1 > = B;

-- Parameterized product:
pair[A, B] = { A first, B second };

-- Inductive linked list:
list[X] = < {} nil, { X car, Y cdr } cons > = Y;
```

Square brackets `[...]` provide deterministic lexical disambiguation: a bare identifier (`X`) represents a variable, whereas a bracketed identifier (`bit[]` or `list[X]`) represents a pattern template application.

5 Linear Canonical Normalization

In a content-addressed system [4], every entity is identified by its cryptographic hash `@hash`. A major problem arises when parameterized patterns enforce equality constraints:

```
pair[A, B]         = { A first, B second };
same_pair[T]       = { T first, T second };
closed_same_pair[] = { T first, T second };
any_pair[]         = { (...) first, (...) second };
```

All four patterns describe a binary record with fields `first` and `second`. If equality constraints were baked into the canonical graph, the compiler would produce four distinct graphs, fracturing global deduplication.

5.1 The Linear Canonical Theorem

K solves this by factoring every pattern into two independent parts:

$$\text{Pattern} = (\mathcal{S}_{\text{canonical}}, \vec{\rho}) \tag{1}$$

1. **Linear Canonical Skeleton ($\mathcal{S}_{\text{canonical}}$)**: Leaves are **separated linearly**. Every open slot is assigned a distinct port $X_0, X_1, \ldots$ in depth-first order.

2. **Parameter Routing Vector ($\vec{\rho}$)**: An external vector routing caller parameters, diagonalized equalities, and wildcards.

The canonical skeleton for all four pair variations is identical:

```
@KqTBNkznjWWwync2Mm94XjCez63neJBJtgYcJJEE5kEm[X0, X1] = {
  X0 first, X1 second
};
```

Their routing vectors capture their specific constraints:

```
pair[A, B]          = @KqTBNkznjWWwync2Mm94XjCez63neJBJtgYcJJEE5kEm[A, B]
same_pair[T]        = @KqTBNkznjWWwync2Mm94XjCez63neJBJtgYcJJEE5kEm[T, T]
closed_same_pair[] = @KqTBNkznjWWwync2Mm94XjCez63neJBJtgYcJJEE5kEm[T, T]
any_pair[]          = @KqTBNkznjWWwync2Mm94XjCez63neJBJtgYcJJEE5kEm[(...), (...)]
```

5.2 Benefits of Linear Decomposition

- **Global Deduplication**: Structurally isomorphic shapes share the exact same `@hash` across all libraries and programs.
- **Affine Multi-Hole Functors**: Skeletons act as pure multi-hole functors $\text{Type}^n \to \text{Type}$.
- **Zero Loss of Type Safety**: When `same_pair[bit[]]` is instantiated, the parameter T routes to both ports, enforcing equality at the call site while preserving the shared linear canonical graph.

6 Variables: Ports, Knots, and Inductive Integrity

Within any pattern template, variables are classified into three orthogonal categories:

1. **External Ports ($\forall$)**: Variables declared in header `[P1, ..., Pn]`, representing open slots supplied by caller arguments.
2. **Internal Existential Variables ($\exists$)**: Free variables within the template body asserting structural equality across multiple fields.
3. **Cycle Knots (μ)**: Variables bound via `= KnotName`, forming directed back-edges that define recursive fixed points.

6.1 Knot Encapsulation and Inductive Integrity

Cycle knots represent the internal topology of recursive definitions. In:

```
list[X] = < {} nil, { X car, Y cdr } cons > = Y;
```

`= Y` binds the root union, and `Y` in field `cdr` refers back to this node, closing the inductive cycle.

To preserve inductive integrity, knots are strictly encapsulated as internal fixed points. Parameter substitution operates exclusively on external ports (`X`), ensuring that recursive cycles remain structurally intact across all instantiations.

6.2 The Knot Invariant

Invariant 6.1 (Knot Invariant)**.** *Formal parameters and cycle knots occupy disjoint namespaces: parameters and knots remain strictly segregated, guaranteeing that internal recursive wiring remains permanently intact.*

7 A Complete Example: Bit Arithmetic

The following self-contained K program illustrates the interplay of documents, relations, and compile-time patterns:

```
-- 1. Patterns (compile-time prototypes)
bit[] = < {} 0, {} 1 >;

-- 2. Constructors
0 = {} |0 ?bit[];
1 = {} |1 ?bit[];
not = < /0 1, 0 >;

-- 3. Half adder: prototype ?{ bit[] a, bit[] b }
half_add = ?{ bit[] a, bit[] b }
  <
    { .a /0 if, { .b sum, 0 carry } then } .then,
    { .b not sum, .b carry }
  >;
```

Here, `?{ bit[] a, bit[] b }` establishes the static input prototype. The two rules concisely handle both cases of `a`:

1. When `a` is `0`: `{ .a /0 if, ... } .then` succeeds, producing sum `b` and carry `0`.
2. When `a` is `1`: choice selects the second branch, producing sum `.b not` and carry `b`.

8 Conclusion

K unifies data, computation, and types into a minimal, content-addressed framework inspired by algebraic process calculi:

- **Documents** are rooted, edge-labeled trees interpreted in the context of schemas.
- **Computation** is partial relations formed by transformation chaining `(f1 f2 ...)`, concurrent choice `< f1, f2, ... >`, and parallel composition `{ f1 label1, f2 label2, ... }`.
- **Types** are patterns acting as static compile-time prototypes and inductive witnesses, with zero runtime overhead.

By decomposing named patterns into pure linear canonical skeletons and parameter routing vectors, K achieves maximal cross-codebase deduplication while preserving author-intended equality constraints. The result is a rigorous, expressive, and elegantly minimal foundation for content-addressed programming. An implementation of K is available on GitHub [6].

A Data Serialization and Enveloped Values

A.1 The Enveloped Value

In K, documents are abstract rooted trees whose structure and interpretation are defined relative to a schema. At execution and transport boundaries, data is represented as an **enveloped value** (or *framed value*): a self-describing pair

$$(S,\, d) \tag{2}$$

where:

- S is a rooted regular pattern graph defining the schema.
- d is a document tree conforming to S.

The schema S serves as the structural authority for the document d. During relation execution, elementary operations propagate and refine this envelope:

- **Product projection `.label`**: extracts the target sub-document and advances the schema to the corresponding field sub-schema.
- **Variant projection `/tag`**: matches the variant tag and advances the schema to the payload sub-schema.
- **Parallel composition `{ f1 l1, ... }`**: constructs a product document paired with the product schema of its branches.
- **Variant constructor `|tag`**: constructs a tagged variant document paired with the union schema containing that tag.

A.2 Self-Hosted Schema Representation

Schemas in K are first-class documents defined directly within the language. The standard core library `core.k` defines the canonical schema for patterns:

```
edge[]    = { string[] label, bits[] target };
edges[]   = < {} nil, { edge[] car, E cdr } cons > = E;
node[]    = <
  {} any,
  edges[] open-product,
  edges[] open-union,
  edges[] closed-product,
  edges[] closed-union
>;
pattern[] = < {} nil, { node[] car, P cdr } cons > = P;
```

A schema is represented as an inductive list of canonical nodes:

- Node 0 designates the root of the schema automaton.
- Edges specify labeled transitions to target node indices, encoded as bit paths (`bits[]`).
- Nodes distinguish between products and unions, as well as open and closed structural contracts.

Because `pattern[]` is a fixed, universally agreed-upon schema constant, it provides the inductive anchor for all K binary serialization.

A.3 The Canonical Wire Stream

The binary wire representation of an enveloped value (S, d) is a two-stage prefix-free bitstream:

$$\text{Stream} = \text{encode}(S : \text{pattern}[])\ \text{encode}(d : S) \tag{3}$$

1. **Schema Header**: The schema S is serialized as a regular K document under the universal singleton schema `pattern[]`.
2. **Document Payload**: The document d is serialized as a prefix-free bitstream interpreted strictly relative to the decoded schema S.

Decoding operates deterministically: the recipient first decodes S using the universal `pattern[]` specification, and then uses S to decode the subsequent payload bitstream d.

A.4 Prefix-Free Payload Encoding

Given the decoded schema S, the document payload d is encoded with maximal compactness:

- **Products**: Field values are encoded sequentially in canonical sorted label order:

$$\text{encode}(d_1)\ \text{encode}(d_2)\ldots\text{encode}(d_k) \tag{4}$$

  Because the schema specifies field identities and ordering, the payload consists exclusively of contiguous field value bits.

- **Variants**: A union alternative is identified by its tag ordinal, encoded in minimal fixed-width bits:

$$w = \lceil \log_2(n) \rceil \tag{5}$$

  where n is the union cardinality, followed immediately by the payload document conforming to the selected alternative.

- **Compact Base Cases**:
  - The empty product `{}` consumes 0 bits.
  - A singleton variant ($n = 1$) consumes 0 bits.
  - A boolean choice ($n = 2$, e.g., `< {} 0, {} 1 >`) consumes exactly 1 bit.

The resulting binary wire format is strictly prefix-free, self-delimiting, and structurally transparent, enabling direct stream processing and lazy sub-document projection.

References

[1] R. Milner. *Communication and Concurrency*. Prentice Hall, 1989.

[2] C. A. R. Hoare. *Communicating Sequential Processes*. Prentice Hall, 1985.

[3] H. Hosoya and B. C. Pierce. Regular expression pattern matching for XML. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 25(2):194–232, 2003.

[4] R. C. Merkle. A digital signature based on a conventional encryption function. In *Advances in Cryptology — CRYPTO '87*, pages 369–378. Springer, 1987.

[5] H. Comon, M. Dauchet, R. Gilleron, C. Löding, F. Jacquemard, D. Lugiez, S. Tison, and M. Tommasi. *Tree Automata Techniques and Applications*. Available online: https://tata.gforge.inria.fr/, 2007.

[6] An implementation of K can be found at GitHub: https://github.com/fraczak/k.